DBMS -- Aggregate Functions & Triggers

Target: Name all 5 aggregate functions in 10s. Explain trigger in 20s with a real use case.

Aggregate Functions in SQL
An aggregate function performs a calculation on multiple rows of a single column and returns a single value.
Used with SELECT, often combined with GROUP BY.

The 5 Aggregate Functions

Function
What it does
NULL handling

COUNT()
Counts rows or non-NULL values
COUNT(*) includes NULLs; COUNT(col) skips NULLs

SUM()
Adds up all values in a column
Skips NULLs

AVG()
Calculates average (mean)
Skips NULLs (avg of non-NULLs only)

MIN()
Returns the smallest value
Skips NULLs

MAX()
Returns the largest value
Skips NULLs

Examples -- Using the EMPLOYEES Table

EMP_ID
NAME
SALARY
DEPT_ID

1
Alice
50000
10

2
Bob
60000
10

3
Carol
45000
20

4
Dave
NULL
20

```sql
-- COUNT() -- total rows including NULLs
SELECT COUNT() FROM EMPLOYEES;              -- 4
-- COUNT(col) -- only non-NULL salary rows
SELECT COUNT(SALARY) FROM EMPLOYEES;        -- 3  (Dave's NULL skipped)
-- SUM -- total salary
SELECT SUM(SALARY) FROM EMPLOYEES;          -- 155000  (NULL skipped)
-- AVG -- average salary (only non-NULLs counted)
SELECT AVG(SALARY) FROM EMPLOYEES;          -- 51666.67  (155000 / 3, not 4)
-- MIN -- lowest salary
SELECT MIN(SALARY) FROM EMPLOYEES;          -- 45000
-- MAX -- highest salary
SELECT MAX(SALARY) FROM EMPLOYEES;          -- 60000
```

AVG trap: AVG skips NULLs in both numerator AND denominator.
AVG(SALARY) = SUM(SALARY) / COUNT(SALARY), not COUNT(*).
Dave's NULL means it's 155000/3 = 51666, not 155000/4 = 38750.

Aggregate with GROUP BY
sql
-- Average salary per department
SELECT DEPT_ID, AVG(SALARY) AS avg_sal, COUNT(*) AS headcount
FROM EMPLOYEES
GROUP BY DEPT_ID;

DEPT_ID
avg_sal
headcount

10
55000
2

20
45000
2

sql
-- Only departments with more than 1 employee earning > 40000
SELECT DEPT_ID, COUNT(*) AS c
FROM EMPLOYEES
WHERE SALARY > 40000   -- filter rows BEFORE grouping
GROUP BY DEPT_ID
HAVING COUNT(*) > 1;   -- filter groups AFTER aggregation

Quick Cheat -- Aggregate + NULL Behaviour
COUNT(*)   counts all rows including NULLs
COUNT(col)   counts non-NULL values only
SUM(col)   adds non-NULLs; returns NULL if all values are NULL
AVG(col)   sum of non-NULLs / count of non-NULLs (NOT total rows)
MIN(col)   smallest non-NULL value
MAX(col)   largest non-NULL value

Triggers
A trigger is a stored procedure that automatically executes when a specific event occurs in the database -- without being called manually.

Think of it as: "If this happens automatically do that."

Trigger Syntax
sql
CREATE TRIGGER [Trigger_name]
[BEFORE | AFTER]
{INSERT | UPDATE | DELETE}
ON [table_name]
[FOR EACH ROW]
[trigger_body]
Parts explained:

Part
Meaning

Trigger_name
Name you give the trigger

BEFORE \| AFTER
Run before or after the event

INSERT \| UPDATE \| DELETE
Which DML event fires the trigger

ON table_name
Which table to watch

FOR EACH ROW
Fire once per affected row (vs once per statement)

trigger_body
SQL code to execute

Trigger Example -- Auto-log every salary update
sql
CREATE TRIGGER log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
  INSERT INTO salary_audit (emp_id, old_salary, new_salary, changed_at)
  VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
END;
Now every time an employee's salary is updated, a row is automatically inserted into salary_audit -- no application code needed.

BEFORE vs AFTER Triggers

Type
When it runs
Common use

BEFORE
Before the DML operation executes
Validate/modify data before saving

AFTER
After the DML operation executes
Logging, auditing, cascading updates

Example use cases:
- BEFORE INSERT auto-format phone number before saving
- AFTER DELETE log which record was deleted and by whom
- BEFORE UPDATE check business rule (e.g., salary can't decrease)
- AFTER INSERT send notification email (via stored proc)

Trigger vs Constraint -- When to Use Which

Constraint
Trigger

Purpose
Enforce data rules
React to events

Example
CHECK (age > 0)
Log every DELETE

Declared where
Table definition
Separate CREATE TRIGGER

Can run custom code?
No
[OK] Yes

Performance
Fast
Slower (extra code runs)

Your 30-Second Script -- Aggregate Functions

"SQL has 5 aggregate functions: COUNT, SUM, AVG, MIN, MAX. They operate on a column across multiple rows and return one value. Important trap: COUNT() counts all rows including NULLs, but COUNT(col) skips NULLs. AVG also skips NULLs -- it divides by the count of non-NULL values, not total rows. They're used with GROUP BY to aggregate per group, and HAVING to filter those groups."*

Your 20-Second Script -- Triggers

"A trigger is a stored procedure that fires automatically when an INSERT, UPDATE, or DELETE happens on a table. You can fire it BEFORE the operation -- to validate or transform data -- or AFTER -- for auditing and logging. Common use case: auto-log every salary change to an audit table."

Follow-Up Questions (Expect These)
Q: What is the difference between COUNT(*) and COUNT(1)?

Functionally identical -- both count all rows. COUNT(1) just evaluates the constant 1 for each row. Modern query optimisers treat them the same.

Q: Can a trigger call another trigger?

Yes -- this is called a cascading trigger. It can cause infinite loops if not designed carefully. Most DBs have a recursion limit.

Q: Can you rollback a trigger's actions?

Yes. If the trigger runs within the same transaction as the DML that fired it, a ROLLBACK undoes both the original DML and the trigger's actions.

Q: What is the difference between a trigger and a stored procedure?

A stored procedure is called explicitly by application code. A trigger fires automatically when a database event occurs -- no manual call needed.